Prisma와 MongoDB 로그인 설치, 설정, 흐름

- 날짜: 2026-11-02
- 대상: Node.js, Next.js 16.4.3, React 19.3.0, Prisma 6.19.3, MongoDB, `lgcns-fe-next`
- 기준은 현재 소스다. 이 글은 패키지 설치, 환경 변수, 사용자 API, 로그인 API, 쿠키까지 이어지는 경로를 설명한다.
- 데이터베이스 비밀번호와 전체 연결 문자열은 적지 않는다. 그 값은 `lgcns-fe-next/.env.local`에만 있다.

1. 설치

작업 디렉터리는 `lgcns-fe-next`다. `package.json`에 `engines` 필드는 없다. 이 환경에서 명령을 실행한 Node.js는 24였다.

```
cd lgcns-fe-next
npm install
npm install prisma@6 @prisma/client@6
npx prisma generate
```

`npm install`은 `package.json`과 lock 파일의 의존성을 받는다. `prisma`와 `@prisma/client`는 둘 다 `dependencies`에 `^6.19.3`으로 들어 있다. CLI와 생성 클라이언트의 버전을 같게 맞춘 것이다.

`npx prisma generate`는 `prisma/schema.prisma`를 읽고 `node_modules/@prisma/client`에 클라이언트를 만든다. 이 명령은 데이터베이스에 접속하지 않는다. 스키마가 바뀌면 다시 실행한다.

컬렉션을 MongoDB에 맞추는 명령은 따로다.

```
set -a && source .env.local && set +a
npx prisma db push
```

Prisma CLI는 기본적으로 `.env`를 읽는다. `.env.local`은 자동으로 읽지 않는다. 그래서 셸에서 `.env.local`을 먼저 불러온 뒤 `db push`를 실행했다. 이 명령이 `spharos` 데이터베이스에 `User` 컬렉션과 이메일 고유 인덱스 `User_email_key`를 만들었다.

개발 서버는 다음이다.

```
npm run dev
```

`package.json`의 `dev` 스크립트는 `next dev`다. Next.js는 프로세스가 시작할 때 `.env.local`을 읽는다. 파일을 만든 뒤에 서버가 이미 떠 있었다면 한 번 다시 시작한다.

2. 기본 설정

환경 변수

lgcns-fe-next/.gitignore의 .env*가 .env, .env.local을 커밋에서 제외한다. 연결 문자열은 .env.local의 DATABASE_URL이다.

형태는 이렇다. 사용자, 비밀번호, 호스트는 실제 값으로 바꾼다. 경로의 데이터베이스 이름은 spharos 다. Prisma용 MongoDB 주소는 경로에 데이터베이스 이름이 있어야 한다.

```
DATABASE_URL="mongodb+srv://USER:PASSWORD@HOST/spharos?retryWrites=true&w=majority"
```

retryWrites=true와 w=majority는 MongoDB 드라이버에 재시도와 쓰기 확인을 알리는 쿼리다. 앱 코드는 이 문자열을 직접 파싱하지 않는다. schema.prisma의 env("DATABASE_URL")가 Prisma에 넘긴다.

스키마

prisma/schema.prisma의 선언은 이렇다.

블록	선언	하는 일
generator client	provider = "prisma-client-js"	prisma generate가 만들 클라이언트 종류
datasource db	provider = "mongodb"	접속 대상을 MongoDB로 고정
datasource db	url = env("DATABASE_URL")	주소는 환경 변수에서만 읽음
model User	MongoDB 컬렉션 User	이메일, 이름, 비밀번호 해시, 생성·수정 시각

id는 String, @id, @default(auto()), @map("_id"), @db.ObjectId다. MongoDB 문서의 _id를 Prisma의 id로 부른다. email은 @unique다. password는 평문이 아니다. createdAt은 @default(now()), updatedAt 은 @updatedAt이다.

Prisma 클라이언트

lib/prisma.ts는 PrismaClient를 하나 만들어 prisma로 내보낸다. 개발 모드에서는 globalThis에 같은 인스턴스를 둔다. next dev가 모듈을 다시 불러도 연결이 계속 늘지 않게 하려는 것이다. 이 파일에는 "use client"가 없다. 서버 모듈이다.

쿠키 이름

component/providers/authCookie.ts는 ACCESS_TOKEN_COOKIE를 "accessToken"이라는 문자열로 내보낸다. 로그인 페이지와 AuthShell이 같은 이름을 쓴다.

3. 파일과 선언

파일	선언	모듈	하는 일
app/(auth)/login/page.tsx	async function handleSubmit + "use server"	서버 컴포넌트	폼 값을 읽어 /api/auth/login을 호출하고, 성공하면 쿠키를 심은 뒤 redirect("/")
component/templates/LoginSection.tsx	function LoginSection	서버 컴포넌트	<form action={handleSubmit}> 안에 클라이언트 폼을 둔다
component/organisms/LoginForm.tsx	파일 첫 줄 "use client"	클라이언트 컴포넌트	입력 상태, Zod 필드 검사, useFormStatus, 로그인 상태면 /로 이동
app/api/auth/login/route.ts	export async function POST	라우트 핸들러	JSON을 검사하고 userService.login을 호출
app/api/user/route.ts	export async function GET, POST	라우트 핸들러	사용자 목록과 생성을 userService에 맡김
service/userService.ts	export const userService	서버 모듈	해시, 검증, list, create, login
lib/prisma.ts	export const prisma	서버 모듈	PrismaClient 한 개
prisma/schema.prisma	model User	Prisma 스키마	MongoDB User 컬렉션
component/providers/AuthShell.tsx	async function AuthShell	서버 컴포넌트	cookies()로 accessToken을 읽어 프로바이더에 넘김
component/providers/AuthProvider.tsx	"use client" + function AuthProvider	클라이언트 컴포넌트	accessToken이 있으면 isLoggedIn이 참
app/layout.tsx	function RootLayout	서버 컴포넌트	Suspense 안에 AuthShell을 둔다. fallback은 null

(auth)는 라우트 그룹이다. URL에는 없다. 화면 주소는 /login이다. API 주소는 /api/auth/login과 /api/user다.

"use server"는 함수를 서버 함수로 표시하는 지시문이다. 로그인 페이지에서는 handleSubmit 본문의 첫 문장이다. "use client"는 그 파일이 클라이언트 컴포넌트 그래프의 시작이라는 지시문이다. 두 지시문은 호이스팅 문법이 아니다.

handleSubmit과 라우트의 POST에 붙은 async는 ECMAScript async 함수다. 호출 결과가 프로미스다. HTML의 <script async>와 같은 속성이 아니다.

4. 로딩 시점

이 파일들은 HTML에 `<script src>`로 적혀 있지 않다. Next가 서버에서 서버 컴포넌트와 라우트 핸들러를 실행하고, `"use client"`로 시작한 그래프만 브라우저 모듈로 보낸다. Prisma와 MongoDB 호출은 그 브라우저 모듈에 없다.

코드	적힌 곳	로딩
`<script src="a.js"> </script>`	HTML. `async`와 `defer`가 없음	동기. 파서가 그 자리에서 멈추고, 받고, 실행한 뒤 파싱을 재개한다
`<script async src="a.js"> </script>`	HTML	받는 즉시 실행한다. 여러 스크립트의 실행 순서는 문서 순서가 아니다
`<script defer src="a.js"> </script>`	HTML	문서를 파싱한 뒤, 문서에 나온 순서로 실행한다
`<script type="module" src="a.js"></script>`	HTML	모듈 그래프. `async`가 없으면 `defer`처럼 파싱이 끝난 뒤 실행한다
`async function handleSubmit`	`login/page.tsx`	스크립트 로딩 속성이 아니다. 호출될 때 프로미스를 반환한다. 실행 위치는 Next 서버다
`export async function POST`	`app/api/auth/login/route.ts`	HTTP `POST`가 올 때 서버에서 실행된다. 브라우저 스크립트가 아니다
`"use client"` 파일	`LoginForm.tsx`	Next가 브라우저용 모듈로 보낸다. 손글씨 `<script>`는 없다

HTML 명세에서 고전 스크립트에 `async`도 `defer`도 없으면 파서를 막는다. `defer`는 문서 파싱이 끝난 뒤 순서대로 실행한다. `type="module"`인 외부 스크립트는 `async` 속성이 없을 때 지연 실행된다. ECMA-262의 async 함수는 그와 별개로, 호출 결과를 Promise로 감싼다.

5. 전체 흐름

사용자 만들기

로그인 컬렉션은 비어 있을 수 있다. 계정을 만드는 경로는 `POST /api/user`다.

1. 라우트가 JSON을 읽는다. `email`은 이메일 형식, `password`는 8자 이상, `name`은 없어도 된다. 검사는 Zod다.
2. `userService.create`가 같은 이메일이 있는지 `prisma.user.findUnique`로 찾는다. 있으면 `409`와 `이미 등록된 이메일입니다.`를 반환한다.
3. 없으면 `hashPassword`가 소금을 만들고 `scryptSync`로 64바이트 해시를 만든다. 저장 문자열은 `소금:해시`다. 16진수다.
4. `prisma.user.create`가 `User` 컬렉션에 문서를 넣는다. 응답의 `data`에는 `password`가 없다. 상태 코드는 `201`이다.

GET /api/user는 userService.list다. findMany의 select에 비밀번호가 없다. 성공 본문은 { success: true, data: users }다.

로그인

1. 브라우저의 /login은 서버 컴포넌트 LoginSection이 그린 <form action={handleSubmit}>이다. method는 함수 액션이라 서버 HTML에서 POST다. 숨은 입력에 서버 액션 식별자가 있다.
2. 폼 안의 LoginForm은 클라이언트 컴포넌트다. 아이디와 비밀번호 상태, 필드 오류, 제출 버튼의 pending을 가진다. 제출되는 값은 React 상태가 아니라 name="userId", name="password"로 만들어진 FormData다.
3. 버튼이 제출을 기다리는 동안 문구는 로그인중이다. useFormStatus는 폼 안의 자식에서만 부모 폼의 pending을 본다.
4. 서버 함수 handleSubmit이 userId와 password를 읽는다. FormData.get의 반환은 string | File | null이다. 텍스트 입력이면 문자열이다.
5. 예전 주소 http://15.164.252.128:8080/api/v1/auth/login 호출은 주석이다. 실행되지 않는다.
6. 서버 함수는 요청 헤더의 호스트로 http://호스트/api/auth/login을 만들어 POST한다. 본문은 { email: userId, password }이고 Content-Type은 application/json이다. 헤더가 없으면 문자열 본문의 Fetch 기본 타입은 text/plain;charset=UTF-8이다.
7. app/api/auth/login/route.ts의 POST가 Zod로 email, password가 비어 있지 않은지 본다. 실패하면 400이다.
8. userService.login이 이메일로 사용자를 찾는다. 없거나 verifyPassword가 거짓이면 401과 이메일 또는 비밀번호가 올바르지 않습니다.다. 없는 사용자와 틀린 비밀번호를 응답 문구로 구분하지 않는다.
9. verifyPassword는 저장된 소금:해시를 나누고, 같은 소금으로 scryptSync를 다시 계산한다. 비교는 timingSafeEqual이다. 길이가 다르면 비교하지 않고 거짓이다.
10. 맞으면 randomBytes(32)를 16진수 문자열로 만들어 accessToken으로 돌려준다. expiresIn은 7200초다. 이 토큰은 MongoDB에 저장하지 않는다. JWT가 아니다.
11. 서버 함수는 data.success가 참이고 accessToken이 문자열이면 cookies().set으로 accessToken 쿠키를 심는다. httpOnly, sameSite: "lax", path: "/", maxAge는 응답의 expiresIn이다. secure는 NODE_ENV가 production일 때만 참이다. 그다음 redirect("/")다.
12. 다음 문서 요청에서 AuthShell이 그 쿠키를 읽어 AuthProvider에 넘긴다. 클라이언트는 isLoggedIn이 참이면 useEffect 안에서 router.replace("/")를 호출한다. 렌더 도중에 router.replace를 호출하지 않는다. 서버에는 location이 없기 때문이다.
13. 프로바이더는 쿠키 값의 존재만 본다. 토큰을 MongoDB나 서명 검사로 다시 확인하는 코드는 없다.

쿠키가 이미 있는 채로 /login이 그려지면, 클라이언트 효과가 메인 페이지로 보낸다. 서버 페이지 함수 자체가 redirect하지는 않는다.

6. 확인한 동작

아래는 구현 직후에 이 앱으로 확인한 결과다. 점검용 계정은 확인 후 지웠고, 당시 `User` 문서는 0건이었다.

요청	결과
`GET /api/user`	`200`, `{ success: true, data: [] }`
`POST /api/user`에 이메일과 8자 이상 비밀번호	`201`, 본문에 비밀번호 없음
`POST /api/auth/login`에 같은 이메일과 비밀번호	`200`, `success: true`, `data.accessToken` 있음
`POST /api/auth/login`에 다른 비밀번호	`401`, 이메일 또는 비밀번호가 올바르지 않습니다.

로그인 화면의 서버 함수는 성공 응답의 `accessToken`이 문자열일 때만 쿠키를 심고 `/`로 이동한다. 실패하면 `redirect`를 호출하지 않는다.

7. 공식 문서

- Node.js, `scryptSync`: https://nodejs.org/api/crypto.html#cryptoscryptsyncpassword-salt-keylen-options
- Node.js, `timingSafeEqual`: https://nodejs.org/api/crypto.html#cryptotimingsafeequala-b
- Node.js, `randomBytes`: https://nodejs.org/api/crypto.html#cryptorandombytessize-callback
- Prisma, MongoDB: https://www.prisma.io/docs/orm/overview/databases/mongodb
- Prisma, 연결 문자열: https://www.prisma.io/docs/orm/reference/connection-urls
- Prisma, `db push`: https://www.prisma.io/docs/orm/reference/prisma-cli-reference#db-push
- Next.js, 환경 변수와 `.env.local`: https://nextjs.org/docs/app/guides/environment-variables
- Next.js, 라우트 핸들러: https://nextjs.org/docs/app/getting-started/route-handlers
- Next.js, `cookies`: https://nextjs.org/docs/app/api-reference/functions/cookies
- Next.js, `redirect`: https://nextjs.org/docs/app/api-reference/functions/redirect
- Next.js, 서버 함수: https://nextjs.org/docs/app/getting-started/updating-data
- MDN, Fetch와 `Content-Type`: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch
- MDN, `FormData.get`: https://developer.mozilla.org/en-US/docs/Web/API/FormData/get
- Fetch 표준, 문자열 본문의 기본 `Content-Type`: https://fetch.spec.whatwg.org/#concept-bodyinit-extract
- ECMA-262, async 함수: https://tc39.es/ecma262/multipage/ecmascript-language-functions-and-classes.html#sec-async-function-definitions
- HTML 명세, `script`의 `async`, `defer`, 모듈: https://html.spec.whatwg.org/multipage/scripting.html#the-script-element
- React, `useFormStatus`: https://react.dev/reference/react-dom/hooks/useFormStatus